



Figure 8: Performance of RetrievalAttention in 1 million Needle-in-a-haystack test.

A.3 PERFORMANCE IN THE EXTREMELY LONG-CONTEXT INFERENCE

Figure 8 shows the evaluation results of RetrievalAttention for extremely long contexts using the model Llama-3-8B-1048K. RetrievalAttention still passes all test cases when ranging the context length from 250K to 1 million, which demonstrates the robustness of our attention-aware indexes.

A.4 DECODING LATENCY ON A100

We test the generality of RetrievalAttention by measuring its performance on a server with one A100 (80GB) and one AMD EPYC 7V13 CPU with 24 cores and 220GB DRAM. We show the token-generation latency of different methods on three models in Table 7. Since the KV cache of full attention is disabled, all prompt tokens need to be recalculated during the decoding, incurring a very high decoding latency. By enabling the KV cache with the PageAttention optimization in vLLM, the decoding latency is significantly reduced. However, vLLM suffers from OOM issue with the increase of context length, which we elaborate further later. Other KV cache dropping or block retrieval methods including StreamingLLM, SnapKV, and InfLLM achieve faster decoding speed, but this is at the expense of a significant drop in model accuracy. In contrast, RetrievalAttention does not compromise generation accuracy while achieving much lower decoding latency than IVF and Flat because of the efficient mitigation of out-of-distribution problem.

We also evaluate how the decoding latency changes when the context length varies from 100K to 1M tokens on Llama-3-8B model and the results can be found in Table 8. To make sure there is enough CPU memory to hold the KV cache and indexes, especially in the 1M context scenario, we use a powerful machine equipped with an AMD EPYC 7V12 CPU with 48 cores and 1.72 TB of memory. The machine is also equipped with the same 80GB A100 GPU. The decoding latency of full attention with KV state re-computation increases quadratically with the context size. With the KV cache enabled in the GPU memory, vLLM starts triggering the OOM issues when the context size is larger than 200K. Static KV dropping methods such as StreamingLLM have no latency increase due to the constant KV cache involved for attention computation. Different from Flat and IVF whose latency numbers are sensitive to context size, RetrievalAttention only has a minor latency increase (8%) when the context size increases $10\times$ from 100K to 1M.

Table 7: Per-token generation latency (s) of 128K context-length on A100.

Methods	Yi-6B	Yi-9B	Llama-3-8B
Full (without cache)	31.61	47.51	33.38
vLLM	0.030	0.044	0.033
StreamingLLM	0.032	0.047	0.031
SnapKV	0.033	0.05	0.033
InfLLM	0.069	0.11	0.068
Flat	0.541	0.802	0.564
IVF	0.309	0.468	0.345
RetrievalAttention	0.150	0.227	0.155

Table 8: Per-token generation latency (s) as context length varies from 100K to 1M.

Methods	100K	200K	500K	1M
Full (without cache)	25.47	83.03	457	1740
vLLM	0.029	0.046	OOM	OOM
StreamingLLM	0.034	0.035	0.032	0.035
SnapKV	0.035	0.035	0.034	0.034
InfLLM	0.082	0.079	0.082	0.084
Flat	0.489	0.871	1.92	3.69
IVF	0.308	0.476	1.032	1.889
RetrievalAttention	0.159	0.167	0.170	0.172

Algorithm 1: RetrievalAttention

Input: Query vector $\mathbf{q}_t \in \mathcal{R}^{1 \times d}$
Data: KV Cache in GPU $\mathbf{K}_{\mathcal{W}}, \mathbf{V}_{\mathcal{W}} \in \mathcal{R}^{|\mathcal{W}| \times d}$
Data: CPU-based Vector Database $\mathcal{H}$
Output: Attention output $\mathbf{o}_t \in \mathcal{R}^{1 \times d}$

```

// Find the predictable KV vectors
1  $\mathcal{W}' \leftarrow \text{PredictActiveTokens}(\dots);$ 
2 for  $\{i | i \in \mathcal{H} \cup \mathcal{W}'\}$  do
3    $\mathcal{H}.\text{remove}(i); \mathcal{W}.\text{insert}(i);$  // move to GPU
4 for  $\{i | i \notin \mathcal{W}' \wedge i \in \mathcal{H}\}$  do
5    $\mathcal{W}.\text{remove}(i); \mathcal{H}.\text{insert}(i);$  // move to CPU
// Attention on GPU
6  $\mathbf{o}_{\mathcal{W}} \leftarrow \text{FlashAttention}(\mathbf{q}_t, \mathbf{K}_{\mathcal{W}}, \mathbf{V}_{\mathcal{W}})$ 
// Attention on CPU
// Search in vector database to retrieve most relevant KV
// vectors
7  $\Omega \leftarrow \text{VectorSearch}(\mathbf{q}_t);$ 
8  $\mathbf{o}_{\Omega} \leftarrow \text{AttentionCPU}(\Omega);$  // Combine partial attention outputs
9  $\mathbf{o}_t = \gamma_1 \cdot \mathbf{o}_{\mathcal{W}} + \gamma_2 \cdot \mathbf{o}_{\Omega};$  // Equation 4, 5

```

B RETRIEVALATTENTION ALGORITHM**B.1 FORMULA OF COMBINING ATTENTION RESULTS FROM THE CPU AND GPU SIDE**

RetrievalAttention partitions the KV vectors for attention into two disjoint sets: predictable ones on GPU (denoted as $\mathcal{W}$) and dynamically retrieved ones on CPU (denoted as Ω).

$$\mathcal{I}_{t,\epsilon} = \mathcal{W} \cup \Omega \quad (3)$$

Attention operation is applied to the two sets of KV vectors separately on CPU and GPU, generating two partial attention outputs (denoted as $\mathbf{o}_{\mathcal{W}}$ and $\mathbf{o}_{\Omega}$, respectively). To guarantee the approximated attention output equals to the attention computation on $\mathcal{I}_{t,\epsilon}$, RetrievalAttention uses a similar idea of FlashAttention (Dao et al., 2022) to combine $\mathbf{o}_{\mathcal{W}}$ and $\mathbf{o}_{\Omega}$ in the following equations:

$$\begin{aligned}
\mathbf{o}_{\mathcal{W}} &= \text{Attn}(\mathbf{q}_t, \mathbf{K}[\mathcal{W},:], \mathbf{V}[\mathcal{W},:]) \\
&= \frac{\sum_{i \in \mathcal{W}} e^{z_i - \tilde{z}_1} \cdot v_i}{\sum_{i \in \mathcal{W}} e^{z_i - \tilde{z}_1}} \\
\mathbf{o}_{\Omega} &= \text{Attn}(\mathbf{q}_t, \mathbf{K}[\Omega,:], \mathbf{V}[\Omega,:]) \\
&= \frac{\sum_{i \in \Omega} e^{z_i - \tilde{z}_2} \cdot v_i}{\sum_{i \in \Omega} e^{z_i - \tilde{z}_2}} \\
\mathbf{o}_t &= \gamma_1 \cdot \mathbf{o}_{\mathcal{W}} + \gamma_2 \cdot \mathbf{o}_{\Omega}
\end{aligned} \quad (4)$$

where $\tilde{z}_1 = \max_{i \in \mathcal{W}} z_i$ and $\tilde{z}_2 = \max_{i \in \Omega} z_i$ are the local maximum dot products in set $\mathcal{W}$ and Ω respectively. And γ_1 and γ_2 are re-scaling factors to guarantee the attention output is the same as that on $\mathcal{I}_{t,\epsilon}$, which are defined as follows:

$$\begin{aligned}
\gamma_1 &= \frac{e^{\tilde{z}_1 - \tilde{z}} \cdot \sum_{i \in \mathcal{W}} e^{z_i - \tilde{z}_1}}{\sum_{i \in \mathcal{I}_{t,\epsilon}} e^{z_i - \tilde{z}}} \\
\gamma_2 &= \frac{e^{\tilde{z}_2 - \tilde{z}} \cdot \sum_{i \in \Omega} e^{z_i - \tilde{z}_2}}{\sum_{i \in \mathcal{I}_{t,\epsilon}} e^{z_i - \tilde{z}}}
\end{aligned} \quad (5)$$

B.2 OVERALL EXECUTION FLOW

Algorithm 1 summarizes the above design of RetrievalAttention and elaborates the procedure in an algorithm. At the beginning of each token generation, RetrievalAttention predicts active KV vectors,

moves them to GPU memory, and computes partial attention using the FlashAttention (Dao et al., 2022) kernel (#1 - #6). In parallel with GPU computation, RetrievalAttention leverages the specially designed vector database to find the most relevant KV vectors to compute attention on CPU (#7 - #8). Finally, RetrievalAttention combines the partial attention outputs on GPU and CPU using #4 and gets the approximated attention output (#9).

C IMPLEMENTATION

RetrievalAttention builds one individual vector index for the KV cache in one attention head. RetrievalAttention has implemented several optimizations to optimize the prompt prefill, accelerate the vector search, and reduce CPU memory usage.

Optimization for the Prefill Phase. During the prefill phase, full attention computation is required to generate the output vector for the next layer of the LLM. Simultaneously, we move the KV vectors to the CPU side for the ANNS index building. To accelerate the overall prefill process, we overlap the cache movement to the CPU with the full attention computation on the GPU in a pipeline manner. To minimize peak GPU memory usage during the prefill phase, attention computation is performed sequentially across multiple attention heads. This approach only slightly impacts the attention computation speed, as longer prompts can fully leverage GPU parallelism with FlashAttention.

Multi-head Parallelism on the CPU side. To speed up the dynamic sparse attention computation on the CPU, we exploit the multi-thread parallelism in vector databases by leveraging the multi-core ability of modern CPU architecture. Specifically, since the computation of different attention heads is independent, we launch multiple threads for parallel searching across different vector indexes to reduce the overall latency on the CPU side. For grouped query attention (GQA) (Ainslie et al., 2023), although multiple query heads could share the same key-value vectors, we observe that the query vectors from different query heads in the same group exhibit different vector distributions. Therefore, we build one vector index for each query head to leverage the specific query distribution of each head.

Minimize the CPU Memory Usage. To reduce CPU memory consumption, the indexes in the same attention group share one copy of KV vectors by only storing the pointers to KV vectors in each index. In the future, we plan to utilize scalar quantization to further compress the KV vectors, implementing an 8-bit quantization in place of the original FP16 format. This compression is promising to reduce memory usage while preserving computational efficiency. Importantly, our initial results demonstrate that this quantization approach does not compromise the inference accuracy, maintaining performance equivalent to the full-precision representation.

D ADDITIONAL RELATED WORK

Sparse Transformers. Since the quadratic complexity of attention has become the bottleneck of LLM efficiency for long context applications, numerous works have studied to design sparse transformers to reduce the computational and memory complexity of the self-attention mechanism. Some works restrict the attention computation to predefined patterns, including sliding windows (Child et al., 2019), dilated windows (Beltagy et al., 2020), or a mixture of different patterns (Zaheer et al., 2020; Ainslie et al., 2020). Some approaches use cluster-based sparsity based on hash value (Kitaev et al., 2020) or KNN algorithms (Bertsch et al., 2024; Mao et al., 2024). These solutions either require pre-training a model from scratch or target limited scenarios like CPU-only, which do not work for our target to out-of-box usage of LLMs on the GPU-CPU architecture. Although some approaches (Xiao et al., 2024a; Ribar et al., 2024) exploit the dynamic sparse nature of LLMs, they often use some estimation using low-rank hidden states or post-statistical approaches, which incurs high overhead but with low accuracy. Moreover, all these approaches have to maintain full KV vectors on GPU with only accelerated inference by reduced memory movement, which does not solve the challenge of commodity GPUs with limited GPU memory.

Additionally, some approaches accelerate the inference by employing dynamically sparse attention patterns (Jiang et al., 2024), separating the prefill and decoding stages (Zhong et al., 2024; Qin et al., 2024), and utilizing sequence parallelism (Jacobs et al., 2023; Liu et al., 2024a). These methods are orthogonal to ours and can be in conjunction with our approach.